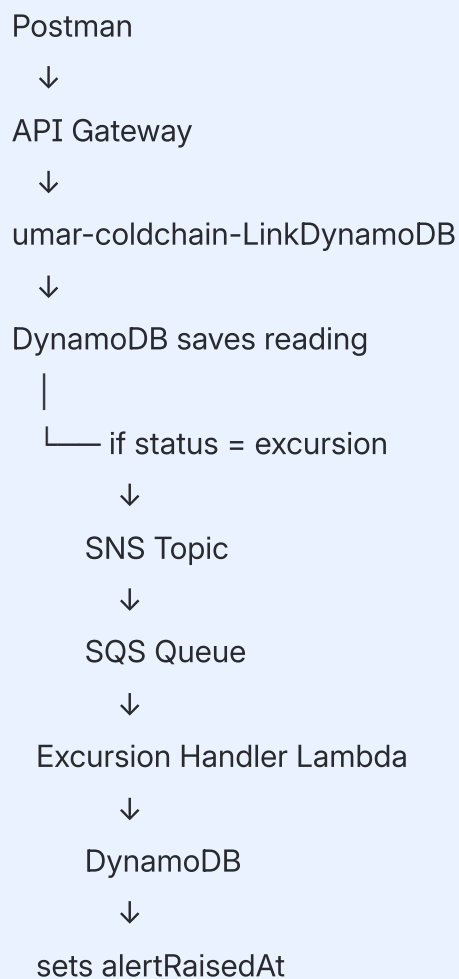


2.6 SQS and SNS: raise the excursion alert

i Okay this ticket is building on the structure we already have set up.
At the moment the structure of your project looks like this:
Postman / Next.js —→ API Gateway —→ Lambda —→
DynamoDB —→ Lambda --→ API Gateway —→ JSON response.

We are now trying to build upon this to make it look something like this:



If processing repeatedly fails:
SQS Queue → Dead Letter Queue

Reasoning for adding SNS and SQS

We are adding an asynchronous alert system around the existing API Gateway, existing create Lambda, and existing DynamoDB table. Up to now, someone still has to look at the table to notice a problem. After this ticket, the system notices on its own.

At the moment if the temperatures drop below a certain amount and the request is sent to the lambda - the database logs it - the API gateway sends back a response to postman. someone still has to manually look the response or the tale manually.

To solve this:

- SNS Topics - will be used to send alerts to these users, E.G the alert in an email (hence the name of the SNS simple NOTIFICATION service)
- SQS Queue - will be used to hold the requests until it can be picked up and ran. E.G. if multiple request have been sent a once will be used for that. (The point of putting SQS between the create Lambda and alert Lambda is that if the alert handler temporarily fails, the original reading has already been saved and the queue keeps the work available for retry.)

SNS = "An excursion happened!"

↓

SQS = "I'll hold this work until the alert Lambda processes it."

Learning objective

A queue and a topic, in one sentence each

From what I have read there are **SNS topics** and **SQS queues**.

SNS topic is a communication channel that uses a push model to broadcast messages to many subscribers at once.

What does SNS topics actually do with requests?? = An Amazon SNS topic acts as a digital megaphone or sorting hub. When an application sends a request (a message) to a topic, the topic instantly copies that message and broadcasts it in parallel to every endpoint, queue, or service subscribed to it.

AI simplified DEF: SNS topic is a channel where one publisher can send a message that is delivered to one or many subscribers.

SQS queue is a temporary storage buffer that holds messages until a single consumer pulls and processes them.

What do SQS queues actually do with requests?? = Amazon SQS (Simple Queue Service) acts as a safe middle layer between parts of an app. When a sender adds a request (called a message), SQS stores it safely on multiple servers. It holds the request until a worker picks it up, processes it, and deletes it. This prevents systems from crashing during heavy traffic.

AI simplified DEF: SQS queue is a place where messages wait until a consumer processes them.

SQS: work is pulled off and processed once. SNS: one message fans out to many subscribers. When would you want each?

SNS = "An excursion happened!"

↓

SQS = "I'll hold this work until the alert Lambda processes it."

Why put a queue in between at all, rather than having the create Lambda raise the alert itself? Think about what happens if raising the alert fails

The point of putting SQS between the create Lambda and alert Lambda is that if the alert handler temporarily fails, the original reading has already been saved and the queue keeps the work available for retry. Lambda/SQS uses an **at-least-once** model, so duplicate processing is possible and your handler must be idempotent.

What is a dead letter queue, and why do you always want one?

A **dead-letter queue (DLQ)** stores messages that repeatedly fail processing so they don't disappear and you can investigate them. SQS uses a redrive policy and `maxReceiveCount` to decide when a failed message is moved to the DLQ.

What does "at least once delivered" mean, and why does it mean your handler has to be safe to run twice?

It means a message may be delivered more than once. This is why you use an SQS queue: to ensure messages are processed reliably and to help avoid concurrent processing issues. On the consumer side, if a user uploads one file and then uploads another later, they will expect the first

one to be uploaded first; SQS helps preserve order (when using FIFO) and prevents two workers from processing the same item simultaneously. Your handler must therefore be safe to run twice (idempotent) to avoid duplicate side effects.

This is why we have SQS queues in place.

Task

i Build the alert flow from the brief:

- When the create Lambda saves a reading with `{{status}}` of `{{excursion}}`, publish a message to an SNS topic
- Subscribe an SQS queue to that topic
- Give the queue a dead letter queue
- Write an excursion handler Lambda that reads the queue, sets `{{alertRaisedAt}}` on that reading, and logs the alert clearly

Then prove all three of these:

- *Happy path* — POST a reading at 12°C, and watch `{{alertRaisedAt}}` appear on the record without you doing anything
- *Failure path* — make the handler throw on purpose. Watch the message end up in the dead letter queue
- *Delivered twice* — send the same message to the queue twice. The handler must not raise a second alert. Fix it so it doesn't

Why that last one matters

In a real compliance system, a duplicate alert is a duplicate audit entry — two records saying the fridge failed once. That's a genuine problem for a pharmacy being inspected, not just untidy data. This is why idempotency gets talked about so much.

Done when

- An out-of-range reading raises its alert automatically, end to end
- The dead letter queue catches your deliberate failure
- The same message arriving twice produces one alert, not two — and you can explain how you made that true
- You've drawn the flow and put it in the worksheet

CONFIGURATION

- i** Build the alert flow from the brief:
- When the create Lambda saves a reading with `{{status}}` of `{{excursion}}`, publish a message to an SNS topic
 - Subscribe an SQS queue to that topic
 - Give the queue a dead letter queue
 - Write an excursion handler Lambda that reads the queue, sets `{{alertRaisedAt}}` on that reading, and logs the alert clearly

Creating the SNS topic

First I went to Simple Notification Service (SNS) → Topics → Create topic. I named the topic “umar-coldchain-excursions” and then created it.

(What is important at this moment is the ARN it gives us, because we will need it for my existing lambda).

Creating the dead letter first queue

The dead letter is an SQS Queue; therefore we need to create an SQS Queue: AWS → SQS → Create queue.

I created a standard SQS Queue named: “umar-coldchain-alert-dlq”.

Creating the main SQS queue

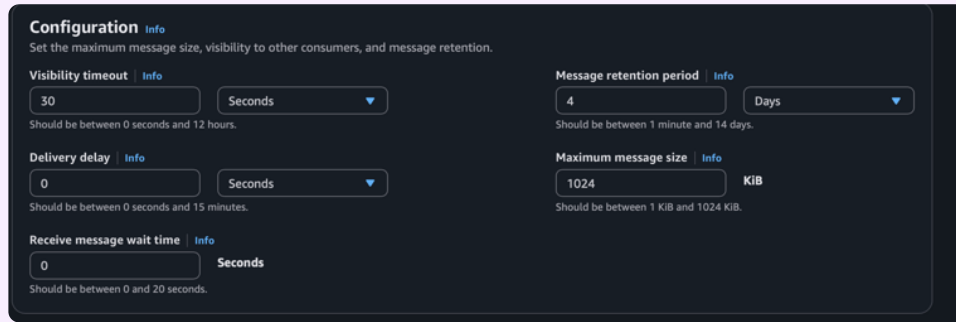
Now I created another standard queue named: “umar-coldchain-alert-queue”.

While creating/editing it, find: Dead-letter queue, enable it and select: “umar-coldchain-alert-dlq”.

The screenshot shows the 'Dead-letter queue' configuration in the AWS console. The title is 'Dead-letter queue - Optional Info'. Below it, a description says 'Send undeliverable messages to a dead-letter queue.' The next section is 'Set this queue to receive undeliverable messages.', with two radio buttons: 'Disabled' and 'Enabled'. The 'Enabled' option is selected. Below this is a 'Choose queue' section with a dropdown menu showing the selected queue: 'arn:aws:sqs:eu-north-1:194442925473:umar-coldchain-alert-dlq'. At the bottom, there is a 'Maximum receives' input field with the value '3' and a note below it: 'Should be between 1 and 1000'.

I changed the maximum receive to a value of three. This means if the Lambda keeps failing to process a message, after repeated receives it will eventually be moved into the DLQ.

NOTE: Visibility timeout



The screenshot shows the 'Configuration' page for an Amazon SQS queue. The 'Visibility timeout' is set to 30 seconds. The 'Message retention period' is set to 4 days. The 'Delivery delay' is set to 0 seconds. The 'Maximum message size' is set to 1024 KiB. The 'Receive message wait time' is set to 0 seconds.

Configuration	Value	Unit	Range
Visibility timeout	30	Seconds	Should be between 0 seconds and 12 hours.
Message retention period	4	Days	Should be between 1 minute and 14 days.
Delivery delay	0	Seconds	Should be between 0 seconds and 15 minutes.
Maximum message size	1024	KiB	Should be between 1 KiB and 1024 KiB.
Receive message wait time	0	Seconds	Should be between 0 and 20 seconds.

If your alert Lambda timeout is: 5 seconds

set the SQS visibility timeout to at least: 30 seconds

AWS recommends the queue visibility timeout be at least six times the Lambda timeout.

So:

- 1 Lambda timeout = 5 seconds
- 2 SQS visibility timeout = 30 seconds

is a simple setup.

(This will make more sense later on when we have actually made the alert Lambda.)

Make the SQS an SNS subscriber

SQS → `umar-coldchain-alert-queue` → Actions → Subscribe to Amazon SNS topic →
Select: umar-coldchain-excursions

Now remember that ARN we pasted into the manual subscription because the name of the SNS wasn't appearing. Then we selected the subscription (view in SNS) and edited it to enable raw message delivery. This is shown in the images below:

Encryption

Amazon SQS key (SSE-SQS)

URL

<https://sqs.eu-north-1.amazonaws.com/194442925473/umar-coldchain-alert-queue>

Dead-letter queue

Enabled

More

Queue policies

Monitoring

SNS subscriptions

Lambda triggers

EventBridge Pipes

Dead-letter queue

Tagging

Encryption

Dead-letter queue

Subscription region

eu-north-1

SNS subscriptions (1)

Info

View in SNS

Delete

Subscribe to Amazon SNS topic

Search subscriptions

Subscription ARN

Topic ARN

arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions:edfb9dad-a631-4ba7-9a28-f08ce3d157bc

arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions

Edit edfb9dad-a631-4ba7-9a28-f08ce3d157bc

Details

Topic

arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions

Protocol

Amazon SQS

Endpoint

arn:aws:sqs:eu-north-1:194442925473:umar-coldchain-alert-queue

Enable raw message delivery

Subscription filter policy - optional

Info

This policy filters the messages that a subscriber receives.

Redrive policy (dead-letter queue) - optional

Info

Send undeliverable messages to a dead-letter queue.

We did this because it will make SNS put something simple into SQS like:

```
{
  "branchId": "BRANCH-001",
  "readingKey": "...",
  "readingId": "...",
  "recordedAt": "...",
  "status": "excursion"
}
```

Instead of wrapping it in a larger SNS message structure.

Changing Lambda to be able to publish excursions

With the existing lambda that we already have (Umar-coldchain-LinkDynamoDB) we need to add SNS functionality to it. So first the code has now changed to the following:

```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2
```

```

3 import {
4     DynamoDBDocumentClient,
5     PutCommand,
6     QueryCommand
7 } from "@aws-sdk/lib-dynamodb";
8
9 import {
10     SNSClient,
11     PublishCommand
12 } from "@aws-sdk/client-sns";
13
14 import { randomUUID } from "node:crypto";
15
16
17 // =====
18 // AWS SETUP
19 // =====
20
21 const REGION = "eu-north-1";
22
23 const TABLE_NAME = "umar_coldchain_database";
24
25 const EXCURSION_TOPIC_ARN =
26     "arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions";
27
28
29 // =====
30 // DYNAMODB SETUP
31 // =====
32
33 const dynamoClient = new DynamoDBClient({
34     region: REGION
35 });
36
37 const dynamo =
38     DynamoDBDocumentClient.from(dynamoClient);
39
40
41 // =====
42 // SNS SETUP
43 // =====
44
45 const sns = new SNSClient({
46     region: REGION
47 });
48
49
50 // =====
51 // BUSINESS RULE
52 // =====
53
54 function getReadingStatus(minTempC, maxTempC) {
55
56     if (
57         !Number.isFinite(minTempC) ||
58         !Number.isFinite(maxTempC)
59     ) {
60         throw new TypeError(
61             "minTempC and maxTempC must both be valid numbers"
62         );
63     }
64
65     if (
66         minTempC < 2 ||
67         maxTempC > 8
68     ) {
69         return "excursion";

```

```

70     }
71
72     return "ok";
73 }
74
75
76 // =====
77 // HTTP RESPONSE HELPER
78 // =====
79
80 function response(statusCode, body) {
81
82     return {
83         statusCode: statusCode,
84
85         headers: {
86             "content-type": "application/json"
87         },
88
89         body: JSON.stringify(body)
90     };
91 }
92
93
94 // =====
95 // DATE HELPER
96 // =====
97
98 function normaliseDate(value) {
99
100     if (!value) {
101         return null;
102     }
103
104     let decodedValue;
105
106     try {
107         decodedValue = decodeURIComponent(value);
108     } catch {
109         decodedValue = value;
110     }
111
112     const timestamp =
113         Date.parse(decodedValue);
114
115     if (Number.isNaN(timestamp)) {
116         return null;
117     }
118
119     return new Date(timestamp).toISOString();
120 }
121
122
123 // =====
124 // FIND READING BY TIMESTAMP
125 // =====
126
127 async function findReadingByTimestamp(
128     branchId,
129     recordedAt
130 ) {
131
132     const result = await dynamo.send(
133         new QueryCommand({
134
135             TableName: TABLE_NAME,
136

```

```

137     KeyConditionExpression:
138         "branchId = :branchId AND begins_with(readingKey,
:recordedAt)",
139
140     ExpressionAttributeValues: {
141         ":branchId": branchId,
142         ":recordedAt": `${recordedAt}#`
143     },
144
145     Limit: 1
146 })
147 );
148
149 return result.Items?.[0];
150 }
151
152
153 // =====
154 // MAIN LAMBDA HANDLER
155 // =====
156
157 export const handler = async (event, context) => {
158
159     console.log("API request received", {
160         requestId: context.awsRequestId,
161         routeKey: event.routeKey,
162         pathParameters: event.pathParameters,
163         queryParameters: event.queryStringParameters
164     });
165
166     try {
167
168         const route = event.routeKey;
169
170
171         const branchId =
172             event.pathParameters?.branchId;
173
174
175         // =====
176         // ROUTE 1
177         // GET /branches/{branchId}/readings
178         // =====
179
180         if (
181             route ===
182             "GET /branches/{branchId}/readings"
183         ) {
184
185             const from =
186                 event.queryStringParameters?.from;
187
188             const to =
189                 event.queryStringParameters?.to;
190
191
192             let keyCondition =
193                 "branchId = :branchId";
194
195
196             const values = {
197                 ":branchId": branchId
198             };
199
200
201             // -----
202             // BOTH FROM AND TO

```

```

203 // -----
204
205 if (from && to) {
206
207     const normalisedFrom =
208         normaliseDate(from);
209
210     const normalisedTo =
211         normaliseDate(to);
212
213
214     if (
215         !normalisedFrom ||
216         !normalisedTo
217     ) {
218
219         return response(400, {
220             error:
221                 "from and to must be valid dates"
222         });
223     }
224
225
226     if (
227         new Date(normalisedFrom) >
228         new Date(normalisedTo)
229     ) {
230
231         return response(400, {
232             error:
233                 "from must be before to"
234         });
235     }
236
237
238     keyCondition +=
239         " AND readingKey BETWEEN :from AND :to";
240
241
242     values[":from"] =
243         `${normalisedFrom}#`;
244
245     values[":to"] =
246         `${normalisedTo}#\uffff`;
247 }
248
249
250 // -----
251 // ONLY FROM
252 // -----
253
254 else if (from) {
255
256     const normalisedFrom =
257         normaliseDate(from);
258
259
260     if (!normalisedFrom) {
261
262         return response(400, {
263             error:
264                 "from must be a valid date"
265         });
266     }
267
268
269     keyCondition +=

```

```

270         " AND readingKey >= :from";
271
272
273         values[":from"] =
274             `${normalisedFrom}#`;
275     }
276
277
278     // -----
279     // ONLY TO
280     // -----
281
282     else if (to) {
283
284         const normalisedTo =
285             normaliseDate(to);
286
287
288         if (!normalisedTo) {
289
290             return response(400, {
291                 error:
292                     "to must be a valid date"
293             });
294         }
295
296
297         keyCondition +=
298             " AND readingKey <= :to";
299
300
301         values[":to"] =
302             `${normalisedTo}#\uffff`;
303     }
304
305
306     const result = await dynamo.send(
307         new QueryCommand({
308
309             TableName: TABLE_NAME,
310
311             KeyConditionExpression:
312                 keyCondition,
313
314             ExpressionAttributeValues:
315                 values,
316
317             ScanIndexForward: false
318         })
319     );
320
321
322     return response(
323         200,
324         result.Items ?? []
325     );
326 }
327
328
329 // =====
330 // ROUTE 2
331 // GET /branches/{branchId}/readings/{recordedAt}
332 // =====
333
334 if (
335     route ===
336     "GET /branches/{branchId}/readings/{recordedAt}"

```

```

337     ) {
338
339         const recordedAt =
340             normaliseDate(
341                 event.pathParameters?.recordedAt
342             );
343
344
345         if (!recordedAt) {
346
347             return response(400, {
348                 error:
349                     "recordedAt must be a valid date"
350             });
351         }
352
353
354         const reading =
355             await findReadingByTimestamp(
356                 branchId,
357                 recordedAt
358             );
359
360
361         if (!reading) {
362
363             return response(404, {
364                 error:
365                     "Reading not found"
366             });
367         }
368
369
370         return response(
371             200,
372             reading
373         );
374     }
375
376
377     // =====
378     // ROUTE 3
379     // POST /branches/{branchId}/readings
380     // =====
381
382     if (
383         route ===
384         "POST /branches/{branchId}/readings"
385     ) {
386
387         let body;
388
389
390         // -----
391         // PARSE JSON
392         // -----
393
394         try {
395
396             body =
397                 JSON.parse(
398                     event.body ?? "{}"
399                 );
400
401         } catch {
402
403             return response(400, {

```

```

404         error:
405             "Request body must be valid JSON"
406     });
407 }
408
409
410 // -----
411 // REQUIRED FIELDS
412 // -----
413
414 const requiredFields = [
415     "recordedAt",
416     "minTempC",
417     "maxTempC",
418     "recordedBy"
419 ];
420
421
422 for (
423     const field of requiredFields
424 ) {
425
426     if (
427         body[field] === undefined ||
428         body[field] === null ||
429         body[field] === ""
430     ) {
431
432         return response(400, {
433             error:
434                 `Missing required field: ${field}`
435         });
436     }
437 }
438
439
440 // -----
441 // TEMPERATURE VALIDATION
442 // -----
443
444 if (
445     !Number.isFinite(body.minTempC) ||
446     !Number.isFinite(body.maxTempC)
447 ) {
448
449     return response(400, {
450         error:
451             "minTempC and maxTempC must be numbers"
452     });
453 }
454
455
456 if (
457     body.maxTempC <
458     body.minTempC
459 ) {
460
461     return response(400, {
462         error:
463             "maxTempC must be greater than or equal to minTempC"
464     });
465 }
466
467
468 // -----
469 // CLIENT CANNOT SET STATUS
470 // -----

```

```

471
472     if (
473         body.status !== undefined
474     ) {
475
476         return response(400, {
477             error:
478                 "status is calculated by the server and must not be
supplied"
479         });
480     }
481
482
483     // -----
484     // TIMESTAMP VALIDATION
485     // -----
486
487     const recordedAt =
488         normaliseDate(
489             body.recordedAt
490         );
491
492
493     if (!recordedAt) {
494
495         return response(400, {
496             error:
497                 "recordedAt must be a valid date"
498         });
499     }
500
501
502     if (
503         new Date(recordedAt) >
504         new Date()
505     ) {
506
507         return response(400, {
508             error:
509                 "recordedAt cannot be in the future"
510         });
511     }
512
513
514     // -----
515     // DUPLICATE CHECK
516     // -----
517
518     const existing =
519         await findReadingByTimestamp(
520             branchId,
521             recordedAt
522         );
523
524
525     if (existing) {
526
527         return response(409, {
528             error:
529                 "A reading already exists for this branch and timestamp"
530         });
531     }
532
533
534     // -----
535     // CALCULATE STATUS
536     // -----

```

```

537
538     const status =
539         getReadingStatus(
540             body.minTempC,
541             body.maxTempC
542         );
543
544
545     // -----
546     // GENERATE ID
547     // -----
548
549     const readingId =
550         randomUUID();
551
552
553     const readingKey =
554         `${recordedAt}#${readingId}`;
555
556
557     // -----
558     // CREATE DYNAMODB ITEM
559     // -----
560
561     const item = {
562
563         branchId: branchId,
564
565         readingKey: readingKey,
566
567         readingId: readingId,
568
569         recordedAt: recordedAt,
570
571         minTempC: body.minTempC,
572
573         maxTempC: body.maxTempC,
574
575         recordedBy: body.recordedBy,
576
577         status: status
578     };
579
580
581     // -----
582     // SAVE READING TO DYNAMODB
583     // -----
584
585     await dynamo.send(
586         new PutCommand({
587
588             TableName:
589                 TABLE_NAME,
590
591             Item:
592                 item
593         })
594     );
595
596
597     console.log(
598         "Reading saved to DynamoDB",
599         {
600             branchId:
601                 branchId,
602
603             readingId:

```

```

604         readingId,
605
606         status:
607             status
608     }
609 );
610
611
612 // =====
613 // IF EXCURSION, PUBLISH TO SNS
614 // =====
615
616 if (
617     status === "excursion"
618 ) {
619
620     const excursionMessage = {
621
622         branchId:
623             branchId,
624
625         readingKey:
626             readingKey,
627
628         readingId:
629             readingId,
630
631         recordedAt:
632             recordedAt,
633
634         minTempC:
635             body.minTempC,
636
637         maxTempC:
638             body.maxTempC,
639
640         status:
641             status
642     };
643
644
645     await sns.send(
646         new PublishCommand({
647
648             TopicArn:
649                 EXCURSION_TOPIC_ARN,
650
651             Message:
652                 JSON.stringify(
653                     excursionMessage
654                 )
655         })
656     );
657
658
659     console.log(
660         "Excursion published to SNS",
661         excursionMessage
662     );
663 }
664
665
666 // -----
667 // SUCCESS RESPONSE
668 // -----
669
670 console.log(

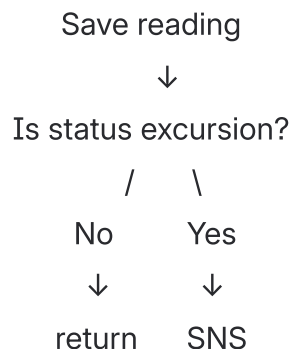
```

```

671     "Reading created",
672     {
673         requestId:
674             context.awsRequestId,
675
676         branchId:
677             branchId,
678
679         readingId:
680             readingId,
681
682         status:
683             status
684     }
685 );
686
687
688     return response(
689         201,
690         item
691     );
692 }
693
694
695 // =====
696 // NO MATCHING ROUTE
697 // =====
698
699     return response(404, {
700         error:
701             "Route not found"
702     });
703
704
705 } catch (error) {
706
707     console.error(
708         "Unhandled API error",
709         error
710     );
711
712
713     return response(500, {
714         error:
715             "Internal server error"
716     });
717 }
718 };

```

So now the code looks like:



Give the existing Lambda SNS permission

The lambda currently only has database permissions; it doesn't have the SNS permissions needed to actually interact and do what we need it to do.

Lambda → `umar-coldchain-LinkDynamoDB` → Configuration → Permissions → Execution role

Open the IAM role and add an inline policy shown below:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "sns:Publish"
      ],
      "Resource": "arn:aws:sns:eu-north-1:194442925473:umar-coldchain-excursions"
    }
  ]
}
```

Create the new alert Lambda

Now we need to create the second Lambda. I named the lambda: “umar-coldchain-excursion-handler”. This lambda has a completely different job compared to the first one.

Your first Lambda: creates readings

Your second Lambda: handles excursion alerts

Give it DynamoDB and SQS permissions

So now the second lambda needs permissions to be able to update an item on the database; and it also needs SQS access as we plan to utilise it.

First for the SQS permissions I add a pre-made permission to the lambda. I attached AWSLambdaSQSQueueExecutionRole. The policy already includes the permissions Lambda needs to consume SQS messages: `sqs:ReceiveMessage` , `sqs:DeleteMessage` , and `sqs:GetQueueAttributes` , plus CloudWatch Logs permissions.

Then for the DynamoDB policy, I created a policy with the following JSON code.

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "dynamodb:UpdateItem"
8       ],
9       "Resource": "arn:aws:dynamodb:eu-north-
10 1:194442925473:table/umar_coldchain_database"
11     }
12   ]
13 }
```

Alert-handler code

Okay now we are going to add the code we need for the alert handler lambda. The code that I inputted is shown below.

```
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 if (message.forceFailure === true) {
3
4   console.error(
5
6     "DELIBERATE_ALERT_FAILURE",
7
8     {
9
10      messageId: record.messageId,
11      readingId: message.readingId
12    }
13  );
14
15  throw new Error(
16
17    "DELIBERATE_ALERT_FAILURE"
18  );
19
20  try {
21
22    const alertRaisedAt =
23      new Date().toISOString();
24
25    await dynamo.send(
```

```

34 new UpdateCommand({
35
36     TableName: TABLE_NAME,
37
38     Key: {
39
40         branchId: message.branchId,
41
42         readingKey: message.readingKey
43
44     },
45
46     UpdateExpression:
47
48     "SET alertRaisedAt = :alertRaisedAt",
49
50     ConditionExpression:
51
52     "attribute_not_exists(alertRaisedAt)",
53
54     ExpressionAttributeValues: {
55
56         ":alertRaisedAt": alertRaisedAt
57
58     }
59
60     })
61
62     );
63
64     console.log("ALERT RAISED", {
65
66         branchId: message.branchId,
67
68         readingId: message.readingId,
69
70         recordedAt: message.recordedAt,
71
72         alertRaisedAt: alertRaisedAt
73
74     });
75
76     } catch (error) {
77
78         // If alertRaisedAt already exists,
79
80         // this message has already been processed.
81
82         if (
83
84             error.name ===
85
86             "ConditionalCheckFailedException"
87
88             ) {
89
90             console.log(
91
92                 "Duplicate message ignored - alert already raised",
93
94                 {
95
96                     branchId: message.branchId,
97
98                     readingId: message.readingId
99
100                 }

```

```

101
102     );
103
104     continue;
105
106     }
107
108     // Any genuine error should make
109     // SQS retry the message.
110
111     throw error;
112
113     }
114
115     }
116
117 }
118 };

```

The really important bit is:

```

1 | ConditionExpression:
2 | "attribute_not_exists(alertRaisedAt)"

```

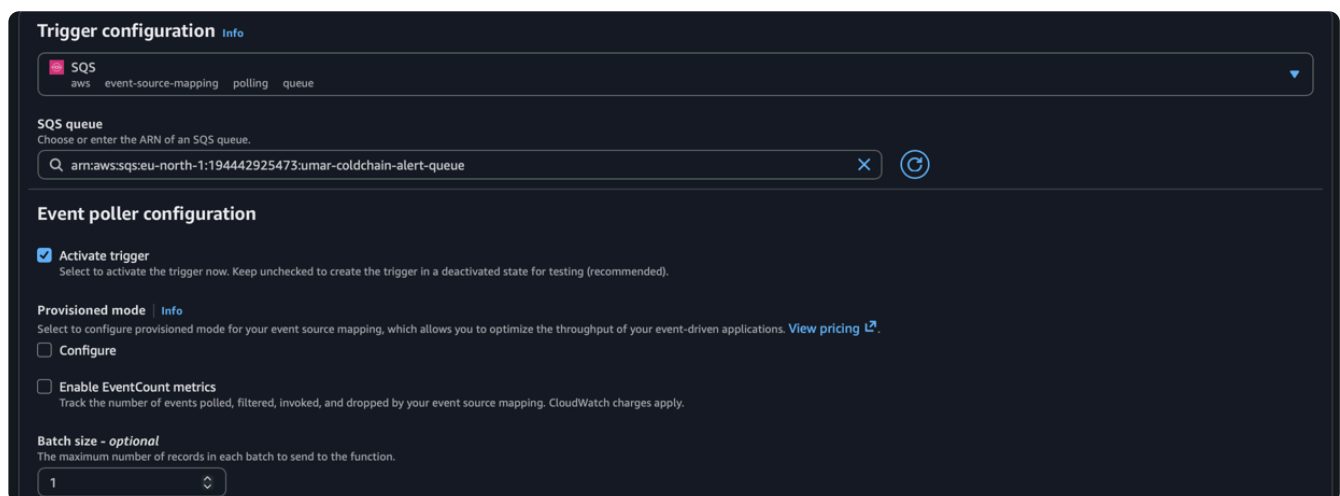
That tells DynamoDB:

Only perform this update if `alertRaisedAt` does not already exist.

DynamoDB supports conditional updates, and conditional writes can be used to make operations idempotent.

Connect SQS to the Lambda

Okay now we need to add the trigger to the lambda, that being the SQS trigger.



The screenshot shows the 'Trigger configuration' page in the AWS Lambda console. At the top, the 'SQS' trigger type is selected. Below this, the 'SQS queue' field contains the ARN 'arn:aws:sqs:eu-north-1:194442925473:umar-coldchain-alert-queue'. The 'Event poller configuration' section shows 'Activate trigger' checked. Under 'Provisioned mode', the 'Configure' checkbox is selected. The 'Batch size - optional' is set to 1.

Then we added the trigger and now all the configuration is done; we can start the testing.

TESTING

i Then prove all three of these:

- *Happy path* — POST a reading at 12°C, and watch `{{alertRaisedAt}}` appear on the record without you doing anything
- *Failure path* — make the handler throw on purpose. Watch the message end up in the dead letter queue
- *Delivered twice* — send the same message to the queue twice. The handler must not raise a second alert. Fix it so it doesn't

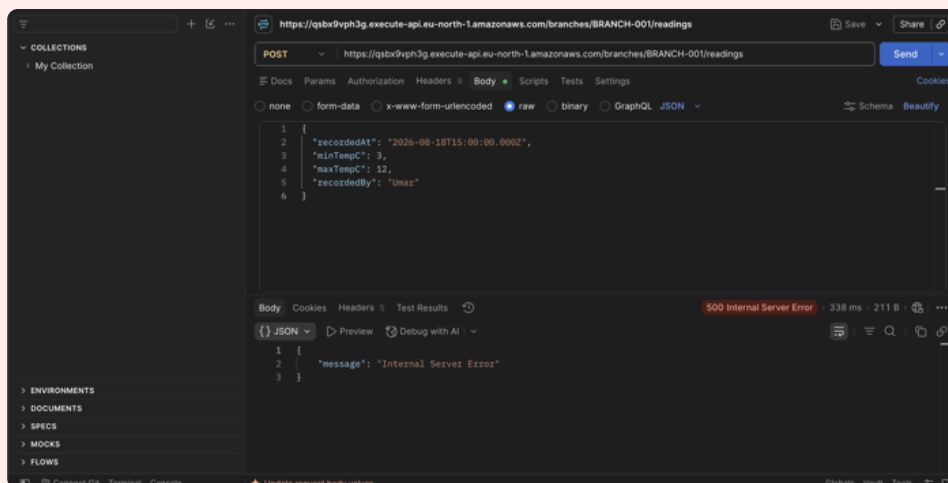
Why that last one matters

In a real compliance system, a duplicate alert is a duplicate audit entry — two records saying the fridge failed once. That's a genuine problem for a pharmacy being inspected, not just untidy data. This is why idempotency gets talked about so much.

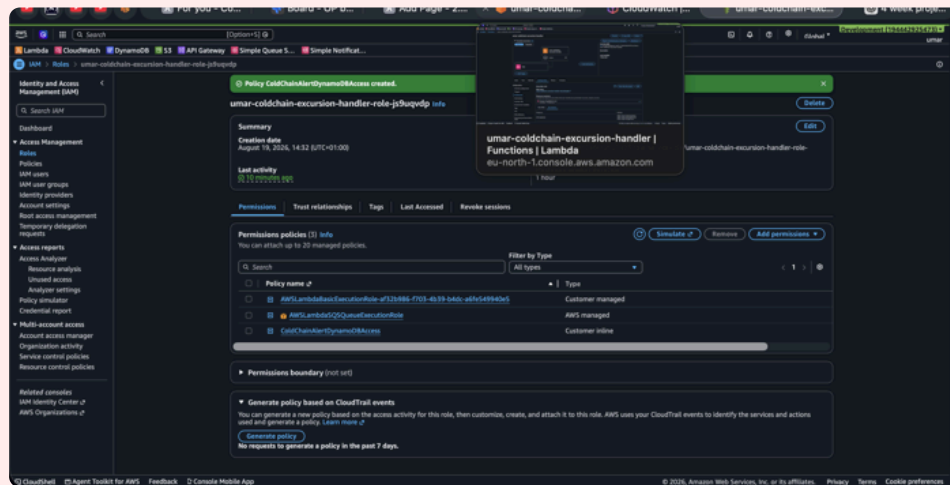
Test 1 — Happy path

“*Happy path* — POST a reading at 12°C, and watch `{{alertRaisedAt}}` appear on the record without you doing anything”

✖ ERROR = there were two errors; the first was when I was running the Postman request I was getting back a 500 instead of 201. This was because the request wasn't getting processed by the lambda code (**umar-coldchain-LinkDynamoDB**) due to improper importation in the code of the SNS. I changed the code and I got the 201 - create I was looking for. (The error can be seen below):

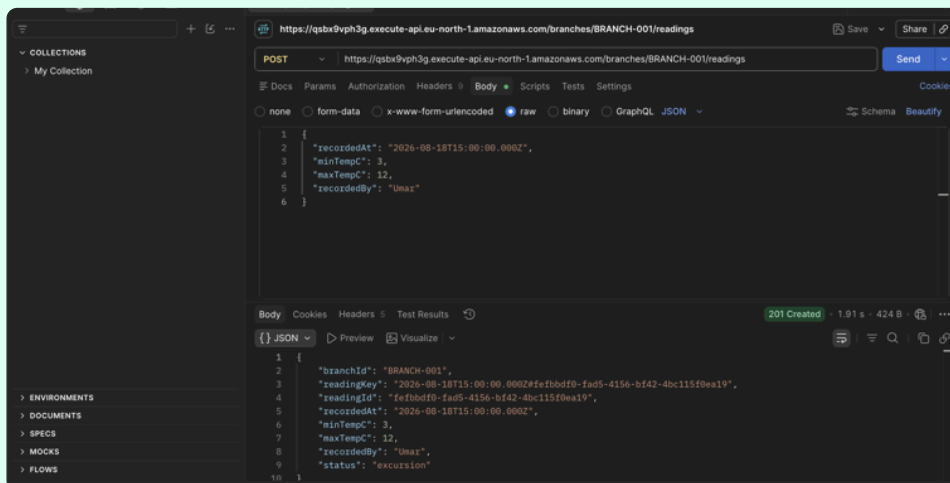


The second issue was the permissions on the (**umar-coldchain-excursion-handler**). The alert column on the database was not showing even though the create request was creating a value in DynamoDB for the excursion readings. After reading the CloudWatch logs I concluded that the lambda didn't actually have access to the database. Therefore it couldn't run its function to make the alert; as a result I had to give it permission as seen in the screenshot below.

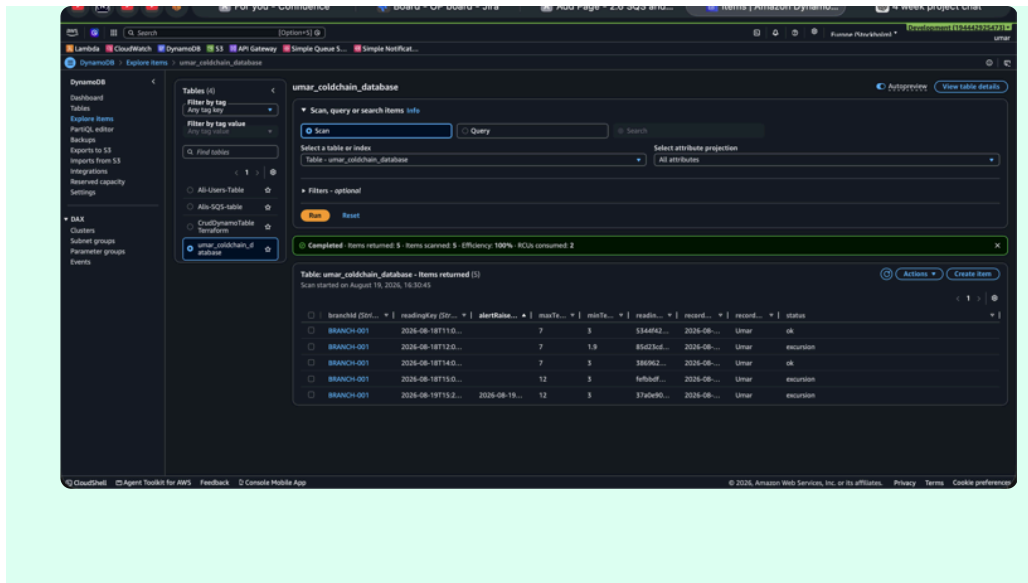


After sorting out the errors above the test passed: the record was created and an alert was created in the database without direct intervention from myself. This can be seen in the screenshots below.

✓ **PASSED** = the screenshot showed



The screenshot below shows the bottom log that I created and shows the alert column on that one.



Test 2 — Failure → DLQ

“Failure path — make the handler throw on purpose. Watch the message end up in the dead letter queue”

Okay now we are making the execution handler fail on purpose and prove the message ends up in the dead-letter queue. The lambda code shouldn't need to be edited again because it contains this code:

```
1 if (message.forceFailure === true) {
2   throw new Error("DELIBERATE_ALERT_FAILURE");
3 }
```

Which should make the lambda support the DLQ but we shall see.

TESTING =

```
1 AWS
2 → SQS
3 → umar-coldchain-alert-queue
4 → Send and receive messages
```

In **Message body**, paste this:

```
1 {
2   "branchId": "BRANCH-001",
3   "readingKey": "FAILURE-TEST",
4   "readingId": "FAILURE-TEST",
5   "recordedAt": "2026-08-19T15:30:00.000Z",
6   "forceFailure": true
7 }
```

Then click:

```
1 | Send message
```

Do **not** click **Poll for messages** on the main queue after sending it. AWS specifically warns that manually viewing/receiving source-queue messages can affect the receive count used for DLQ redrive.

What should now happen automatically is:

```
1 | Main SQS queue
2 |   ↓
3 | Excursion Lambda receives message
4 |   ↓
5 | forceFailure = true
6 |   ↓
7 | Lambda deliberately throws ✗
8 |   ↓
9 | message is NOT deleted
10 |  ↓
11 | 30-second visibility timeout
12 |  ↓
13 | message becomes available again
14 |  ↓
15 | Lambda retries
16 |  ↓
17 | fails again
18 |  ↓
19 | eventually moves to DLQ
```

Because SQS only deletes the batch after successful Lambda processing, failed processing causes the message to become available for retry.

If your main queue has:

```
1 | Maximum receives = 3
2 | Visibility timeout = 30 seconds
```

you may see several attempts about 30 seconds apart before SQS sends it to the DLQ.

`maxReceiveCount` controls how many times a source-queue message can be received before redrive to the DLQ.

TO SEE IF DLQ HAS WORKED =

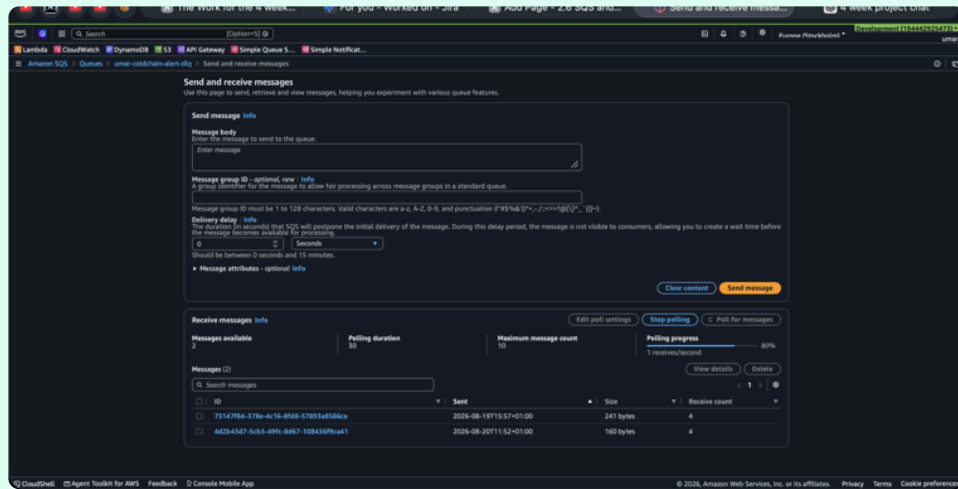
```
1 | SQS
2 | → umar-coldchain-alert-dlq
3 | → Send and receive messages
4 | → Poll for messages
```

This time **yes, click Poll for messages**.

You want to see **1 message** in the DLQ.

✓ **PASSED** = the screenshot showed the message we were looking for in the DLQ. If you are confused as to why there are two messages

the second message is from an old failed request; so we can just ignore that one.



Test 3 — Delivered twice

“Delivered twice — send the same message to the queue twice. The handler must not raise a second alert. Fix it so it doesn't”

What we are aiming for in this test is the handler will be sent the exact same alert message to SQS twice. The first one may raise the alert, but the second one must not create or overwrite another alert.

This test is a slightly longer process than the other ones so I have split it into steps:

Step 1 = In DynamoDB, choose one of your older rows where:

```
1 | status = excursion
```

but there is currently **no** alertRaisedAt .

BRANCH-001	2026-08-18T15:0...	12	3	fefbdf...	2026-08-...	Umar	excursion
------------	--------------------	----	---	-----------	-------------	------	-----------

Open that item and copy these four exact values:

```
1 | "branchId": "BRANCH-001",
2 | "readingKey": "2026-08-18T15:00:00.000Z#fefbbdf0-fad5-4156-bf42-
  | 4bc115f0ea19",
3 | "readingId": "fefbbdf0-fad5-4156-bf42-4bc115f0ea19",
4 | "recordedAt": "2026-08-18T15:00:00.000Z",
5 | "status": "excursion"
```

Step 2 =

Go to:

```
1 | SQS
2 | → umar-coldchain-alert-queue
3 | → Send and receive messages
```

In **Message body**, enter:

```
1 | {
2 |   "branchId": "PASTE_BRANCH_ID",
3 |   "readingKey": "PASTE_FULL_READING_KEY",
4 |   "readingId": "PASTE_READING_ID",
5 |   "recordedAt": "PASTE_RECORDED_AT",
6 |   "status": "excursion"
7 | }
```

Then click **Send message**.

Step 3 = Wait a few seconds and refresh that exact DynamoDB reading.

It should now gain:

```
1 | alertRaisedAt = 2026-08-20T...
```

Make a note of that timestamp.

That was **delivery number 1**.

✓ **PASSED =**

☐	BRANCH-001	2026-08-18T14:0...		7	3	386962...	2026-08-...	Umar	ok
☐	BRANCH-001	2026-08-18T15:0...	2026-08-20...	12	3	fefbddf...	2026-08-...	Umar	excursion
☐	BRANCH-001	2026-08-19T15:2...	2026-08-19...	12	3	37a0e90...	2026-08-...	Umar	excursion

Step 4 = Click Send message again.

That is delivery number 2.

Now refresh DynamoDB again.

The important result is:

```
1 | alertRaisedAt
```

should still have the exact same timestamp as before.

It should not be replaced with a newer timestamp.

First timestamp is: 2026-08-20T12:38:29.652Z

Attributes

Add new attribute ▼

Attribute name	Value	Type	Remove
branchId - Partition key	BRANCH-001	String	
readingKey - Sort key	2026-08-18T15:00:00.000Z#fefbbdf0-fad5-4156-bf42-4bc115f0ea19	String	
alertRaisedAt	2026-08-20T12:38:29.652Z	String	Remove
maxTempC	12	Number	Remove
minTempC	3	Number	Remove
readingId	fefbbdf0-fad5-4156-bf42-4bc115f0ea19	String	Remove
recordedAt	2026-08-18T15:00:00.000Z	String	Remove
recordedBy	Umar	String	Remove
status	excursion	String	Remove

CancelSaveSave and close

Ran the message again and it was exactly the same.

Second timestamp is: 2026-08-20T12:38:29.652Z

Attributes

Add new attribute ▼

Attribute name	Value	Type	Remove
branchId - Partition key	BRANCH-001	String	
readingKey - Sort key	2026-08-18T15:00:00.000Z#fefbbdf0-fad5-4156-bf42-4bc115f0ea19	String	
alertRaisedAt	2026-08-20T12:38:29.652Z	String	Remove
maxTempC	12	Number	Remove
minTempC	3	Number	Remove
readingId	fefbbdf0-fad5-4156-bf42-4bc115f0ea19	String	Remove
recordedAt	2026-08-18T15:00:00.000Z	String	Remove
recordedBy	Umar	String	Remove
status	excursion	String	Remove

CancelSaveSave and close

This proves that

Prove the final requirement:

✔ Same message delivered twice

- ✓ Only one alert raised
- ✓ alertRaisedAt wasn't overwritten
- ✓ Handler is idempotent

⚠ Questions / Queries / Pushback

Now we are ready for the demo (ask Luke how to do it)